Parcial Algoritmos y Programación II - 7541 Curso Calvo (1º instancia) 27 de mayo de 2016

Padrón: **Apellido y Nombre:**

Punteros: APROBADO - DESAPROBADO

1) Indicar la salida por pantalla y escribir las sentencias necesarias para liberar correctamente la memoria.

```
int main(){

  int *A, *C, *F;
  int **B;
  char *D, *E;
  char G;
  int H;

  H = 68;
  G = 'A';
  A = &H;
  D = &G;
  B = &A;
  cout << *A <<  *D << **B << endl;

  C = new int[3];
  F = C + 2;
  C[1] = H;
  (*A) = 70;
  (*C) = H;
  (*F) = 68;
  C[2]++;
  cout << C[2] << C[1] << C[0] << endl;

  (**B) = C[0] + 1;
  A = C + 2;
  (**B) = H;
  C[0] = *A;
  (*F) = A[0] - 3;
  cout << C[2] << C[1] << C[0] << endl;

  E = (char*) A;
  C[1] = (**B) - 4;
  cout << (*D) << (*E) << G << endl;

  // liberar la memoria
  return 0;
}
```

2. Implementar para la clase Cola<T> las primitivas, indicando pre y post condiciones:

```
void acolar(T elemento);
T desacolar();
```

3. Implementar el método `'facturarAgrupandoPorConsorcio'` de la clase `'Facturador'` a partir de las siguientes especificaciones:

```
class Facturador {

  public:

    /* post: procesa 'consumos' para crear y devolver las Facturas correspondientes, considerando las siguientes reglas:
     *       1) Todos los consumos en la misma dirección son de un consorcio y se deben unificar en una única Factura,
     *          (con nombre "Consorcio de <direccion>") salvo que sea de tipo COMERCIAL. Una por el consorcio y una por
     *          cada consumo COMERCIAL.
     *       2) Si el consumo es SUBSIDIADO tiene un descuento del 50%.
     *       3) Si el consumo es COMERCIAL tiene un adicional de 'recargoComercial' x superficie.
     */
    List<Factura*>* facturarAgrupandoPorConsorcio(List<Consumo*>* consumos, float precioPorMetro2, float recargoComercial);

};
```

```
class Factura {

 public:

  /* post: factura inicializada con monto $ 0,00
   */
  Factura(string nombreCliente, string direccion);

  string obtenerNombreCliente();

  void cambiarCliente(string nombre);

  string obtenerDireccion();

  /* post: incrementa el monto total de la factura con el monto dado.
   */
  void acumular(float monto);

  float obtenerTotal();

};
```

```
class Consumo {

 public:

  Consumo(string cliente, string direccion,
          TipoDeConsumo tipo, float superficie);

  string obtenerCliente();

  string obtenerDireccion();

  TipoDeConsumo obtenerTipo();

  float obtenerSuperficie();

};

enum TipoDeConsumo {

  DOMICILIARIO; COMERCIAL; SUBSIDIADO;
};
```

4. Diseñar la especificación e implementar el TDA **Celular**. Debe proveer operaciones para:

- Bloquearlo.
- Desbloquearlo.
- Recibir una llamada (desbloquea automáticamente).
- Realizar una llamada (tiene que estar desbloqueado).
- Finalizar una llamada, indicando la duración en minutos.
- Conocer el estado de la batería: 0 a 100%. Cada minuto de llamada consume 0,1%.
- Devolver la cantidad de llamadas recibidas y realizadas.
- Devolver la duración máxima de una llamada.

Los alumnos que tienen aprobado el parcialito de punteros no deben realizar el ejercicio 1.
Para aprobar es necesario tener al menos el 60% de cada uno de los ejercicios correctos y completos.
Para cada método escribir pre y post condición, si recibe argumentos y cuáles, y si retorna un dato y cuál. De faltar ésto, se considerará el código incompleto.
Duración del examen : 3 horas

Solución 1:

68A68
696870
686871
ADA

Los alumnos que tienen aprobado el parcialito de punteros no deben realizar el ejercicio 1.
Para aprobar es necesario tener al menos el 60% de cada uno de los ejercicios correctos y completos.
Para cada método escribir pre y post condición, si recibe argumentos y cuáles, y si retorna un dato y cuál. De faltar ésto, se considerará el código incompleto.
Duración del examen : 3 horas